

Cost Optimization tradeoffs

When you design a workload to maximize return on investment (ROI) under financial constraints, you first need clearly defined functional and nonfunctional requirements. A work and effort prioritization strategy is essential. The foundation is a team that has a strong sense of financial responsibility. The team should have a strong understanding of available technologies and their billing models.

After you understand the ROI of a workload, you can start improving it. Consider how decisions based on the [Cost Optimization design principles](#) and the recommendations in the [Design review checklist for Cost Optimization](#) might influence the goals and optimizations of other Well-Architected Framework pillars. For cost optimization, it's important to avoid focusing on a cheaper solution. Choices that focus only on minimizing spending can undermine your workload's business goals and reputation. This article describes example tradeoffs that a workload team might encounter when setting targets, designing, and planning operations for cost optimization.

Cost Optimization tradeoffs with Reliability

The cost of a service disruption must be measured against the cost of preventing or recovering from one. If the cost of disruptions exceeds the cost of reliability design, you should invest more to prevent or mitigate disruptions. Conversely, the cost of the reliability efforts might be more than the cost of a disruption, including factors like compliance requirements and reputation. You should consider strategic divestment in reliability design only in this scenario.



Tradeoff: Reduced resiliency. A workload incorporates resiliency measures to attempt to avoid and withstand specific types and quantities of malfunction.

- To save money, the workload team might underprovision a component or overconstrain its scaling, making the component more likely to fail during sudden spikes in demand.
- Consolidating workload resources (*increasing density*) for cost optimization makes individual components more likely to fail during spikes in demand and during maintenance operations like updates.
- Removing components that support resiliency design patterns, like a message bus, and creating a direct dependency reduces self-preservation capabilities.

- Saving money by reducing redundancy can limit a workload's ability to handle concurrent malfunctions.
- Using budget SKUs might limit the maximum service-level objective (SLO) that the workload can reach.
- Setting hard spending limits can prevent a workload from scaling to meet legitimate demand.
- Without reliability testing tools or tests, the reliability of a workload is unknown, and it's less likely to meet reliability targets.



Tradeoff: Limited recovery strategy. A workload that's reliable has a tested incident response and recovery plan for disaster scenarios.

- Reduced testing or drilling of a workload's disaster recovery plan might affect the speed and effectiveness of recovery operations.
- Creating or retaining fewer backups decreases possible recovery points and increases the chance of losing data.
- Choosing a less expensive support contract with technology partners might increase workload recovery time due to potential delays in technical assistance.



Tradeoff: Increased complexity. A workload that uses straightforward approaches and avoids unnecessary or overengineered complexity is generally easier to manage in terms of reliability.

- Using cost-optimization cloud patterns can add new components, like a content delivery network (CDN), or shift duties to edge and client devices that a workload must provide reliability targets for.
- Event-based scaling can be more complicated to tune and validate than resource-based scaling.
- Reducing data volume and tiering data through data lifecycle actions, possibly in conjunction with implementing aggregated data points before a lifecycle event, introduces reliability factors to consider in the workload.

- Using different regions to optimize cost can make management, networking, and monitoring more difficult.

Cost Optimization tradeoffs with Security

The cost of a compromise to confidentiality, integrity, or availability must always be balanced against the cost of preventing it. A security incident can cause significant financial, legal, and reputational harm. Investing in security mitigates risk, and that investment must be measured against the cost of experiencing the risk. As a rule, don't compromise on security to gain cost optimizations that are below the point of responsible and agreed upon risk mitigation.

Optimizing security costs by rightsizing solutions is an important optimization practice, but be aware of tradeoffs like the following when doing so.



Tradeoff: Reduced security controls. Security controls are established across multiple layers, sometimes redundantly, to provide defense in depth.

One cost optimization tactic is to look for ways to remove components or processes that accrue unit or operational costs. Removing security components like the following examples for the sake of saving money impacts security. You need to carefully perform a risk analysis on this impact.

- Reducing or simplifying authentication and authorization techniques compromises the *verify explicitly* principle of Zero Trust architecture. Examples of these simplifications include using a basic authentication scheme like preshared keys rather than investing time to learn industry OAuth approaches, or using simplified role-based access control assignments to reduce management overhead.
- Removing encryption in transit or at rest to reduce costs on certificates and their operational processes exposes data to potential integrity or confidentiality breaches.
- Removing or reducing security scanning, inspection tooling, or security testing because of the associated cost and time investment can directly affect the confidentiality, integrity, or availability that those tools and tests are intended to protect.
- Reducing the frequency of security patching to save operational time on cataloging and applying patches affects a workload's ability to address evolving threats.
- Removing network controls like firewalls might lead to a failure to block malicious inbound and outbound traffic.



Tradeoff: Increased workload surface area. The Security pillar prioritizes a reduced and contained surface area to minimize attack vectors and the management of security controls.

Cloud design patterns that optimize costs sometimes necessitate the introduction of additional components. These additional components increase the surface area of the workload. The components and the data within them must be secured, possibly in ways that aren't already used in the system. These components and data are often subject to compliance. Examples of patterns that can introduce components include:

- Using the Static Content Hosting pattern to offload data to a new CDN component.
- Using the Valet Key pattern to offload processing and secure resource access to client compute.
- Using the Queue-Based Load Leveling pattern to smooth costs by introducing a message bus.



Tradeoff: Removed segmentation. The Security pillar prioritizes strong segmentation to support the application of targeted security controls and to control the blast radius.

Sharing resources, for example in multitenancy situations or co-locating multiple applications on a shared application platform, is an approach for reducing costs by increasing density and reducing the management surface. This increased density can lead to security concerns like these:

- Lateral movement between components that share resources is easier. A security event that compromises the availability of the application platform host or an individual application also has a larger blast radius.
- Co-located resources might share a workload identity and have less meaningful audit trails in access logs.
- Network security controls must be broad enough to cover all co-located resources. This configuration potentially violates the principle of least privilege for some resources.
- Co-locating disparate applications or data on a shared host can lead to extending compliance requirements and security controls to applications or data that would otherwise be out of scope. This broadening of scope necessitates additional security scrutiny and auditing effort on the co-located components.

Cost Optimization tradeoffs with Operational Excellence



Tradeoff: Compromised software development lifecycle (SDLC) capacities. A workload's SDLC process provides rigor, consistency, specificity, and prioritization to change management in a workload.

- Reducing testing efforts to save time and the cost associated with test personnel, resources, and tooling can result in more bugs in production.
- Delaying paying back technical debt to focus personnel efforts on new features can lead to slower development cycles and overall reduced agility.
- Deprioritizing documentation to focus personnel efforts on product development can lead to longer onboarding time for new employees, impact the effectiveness of incident response, and compromise compliance requirements.
- A lack of investment in training leads to stagnated skills, reducing the team's ability to adopt newer technologies and practices.
- Removing automation tooling to save money can result in personnel spending more time on the tasks that are no longer automated. It also increases the risk of errors and inconsistencies.
- Reducing planning efforts, like scoping and activity prioritization, to cut expenses can increase the likelihood of rework due to vague specifications and poor implementation.
- Avoiding or reducing continuous improvement activities, like retrospectives and after-incident reports, to keep the workload team focused on delivery can create missed opportunities to optimize routine, unplanned, and emergency processes.



Tradeoff: Reduced observability. Observability is necessary to help ensure that a workload has meaningful alerting and successful incident response.

- Decreasing log and metric volume to save on storage and transfer costs reduces system observability and can lead to:
 - Fewer data points for creating alerts related to reliability, security, and performance.
 - Coverage gaps in incident response activities.

- Limited observability into interactions or boundaries related to security or compliance.
- Cost optimization design patterns can add components to a workload, increasing its complexity. The workload monitoring strategy must include those new components. For example, some patterns might introduce flows that span multiple components or shift processes from the server to the client. These changes can increase the complexity of correlating and tracking information.
- Reduced investment in observability tooling and the maintenance of effective dashboards can decrease the ability to learn from production, validate design choices, and inform product design. This reduction can also hamper incident response activities and make it harder to meet the recovery time objective and SLO.



Tradeoff: Deferred maintenance. Workload teams are expected to keep code, tooling, software packages, and operating systems patched and up to date in a timely and orderly way.

- Letting maintenance contracts with tooling vendors expire can result in missed optimization features, bug resolutions, and security updates.
- Increasing the time between system patches to save time can lead to missed bug fixes or a lack of protection against evolving security threats.

Cost Optimization tradeoffs with Performance Efficiency

The Cost Optimization and Performance Efficiency pillars both prioritize maximizing a workload's value. Performance Efficiency emphasizes meeting performance targets without spending more than necessary. Cost Optimization emphasizes maximizing the value produced by a workload's resources without exceeding performance targets. As a result, Cost Optimization often improves Performance Efficiency. However, there are Performance Efficiency tradeoffs associated with Cost Optimization. These tradeoffs can make it harder to reach performance targets and hinder ongoing performance optimization.



Tradeoff: Underprovisioned or underscaled resources. A performance-efficient workload has enough resources to serve demand but doesn't have excessive unused overhead, even when usage patterns fluctuate.

- Reducing costs by downsizing resources can deprive applications of resources. The application might not be able to handle significant usage pattern fluctuations.
- Limiting or delaying scaling to cap or reduce costs might result in insufficient supply to meet demand.
- Autoscale settings that scale down aggressively to reduce costs might leave a service unprepared for sudden spikes in demand or cause frequent scaling fluctuations (flapping).



Tradeoff: Lack of optimization over time. Evaluating the effects of changes in functionality, changes in usage patterns, new technologies, and different approaches on the workload is one way to try to increase efficiency.

- Limiting the focus on developing expertise in performance optimization in order to prioritize delivery can cause missed opportunities for improving resource usage efficiency.
- Removing access performance testing or monitoring tools increases the risk of undetected performance issues. It also limits the ability for a workload team to execute on measure/improve cycles.
- Neglecting areas prone to performance degradation, like data stores, can gradually deteriorate query performance and elevate overall system usage.

Related links

Explore the tradeoffs for the other pillars:

- [Reliability tradeoffs](#)
- [Security tradeoffs](#)
- [Operational Excellence tradeoffs](#)
- [Performance Efficiency tradeoffs](#)

Last updated on 04/27/2026